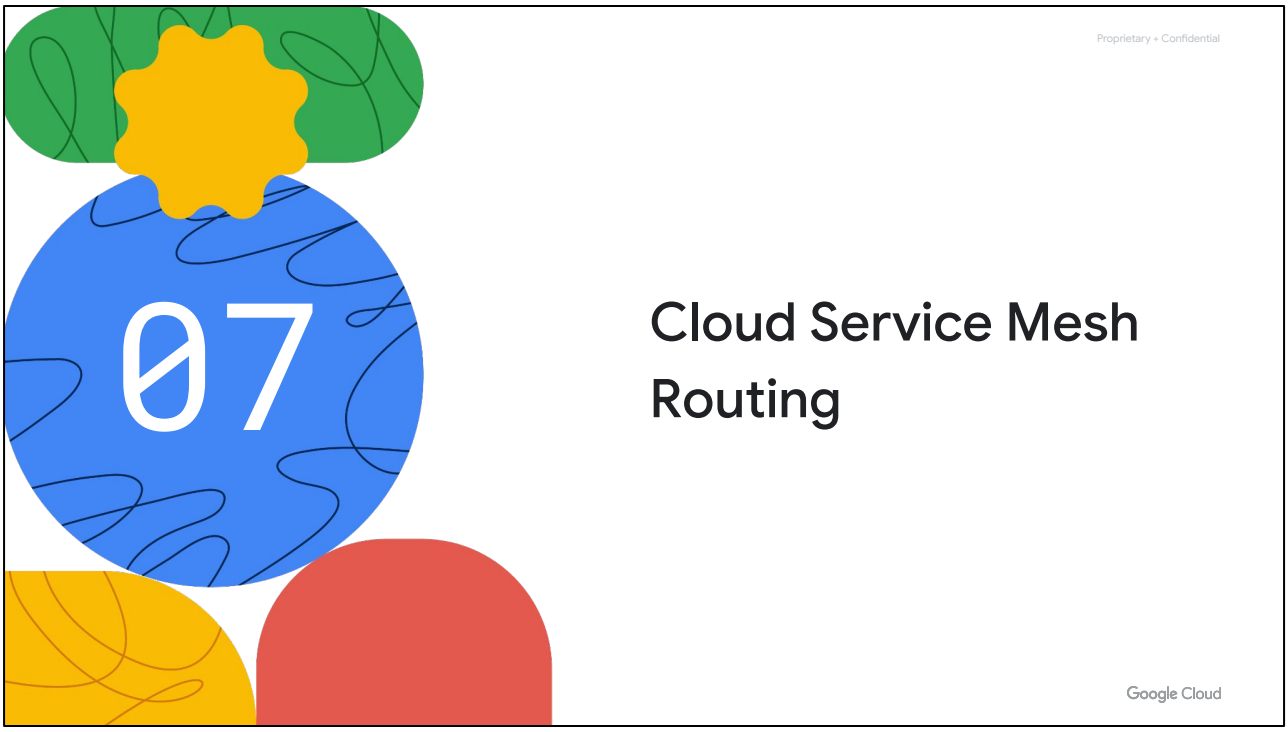


Welcome to the Cloud Service Mesh routing module.

Proprietary + Confidential

Cloud Service Mesh Routing

01 Configuring Cloud Service Mesh with Istio API resources

02 Configuring VirtualService and DestinationRule

03 Configuring ServiceEntry

04 Configuring a Gateway

05 Configuring a WorkloadEntry and WorkloadGroup

06 Network resilience and testing

Google Cloud

In this section, titled “Cloud Service Mesh Routing,” you learn to:

- Explain how Cloud Service Mesh learns the network from Kubernetes.
- Configure traffic behavior using VirtualService and DestinationRule.
- Manage traffic for services running outside the mesh using ServiceEntry.
- Configure Gateways to manage inbound and outbound traffic to the mesh.
- Explore how WorkloadEntry and WorkloadGroup onboards and manages non-Kubernetes workloads.
- Evaluate network resilience and test it through fault injection and traffic mirroring.

Proprietary + Confidential

Cloud Service Mesh Routing

01 Configuring Cloud Service Mesh with Istio API resources

02 Configuring VirtualService and DestinationRule

03 Configuring ServiceEntry

04 Configuring a Gateway

05 Configuring a WorkloadEntry and WorkloadGroup

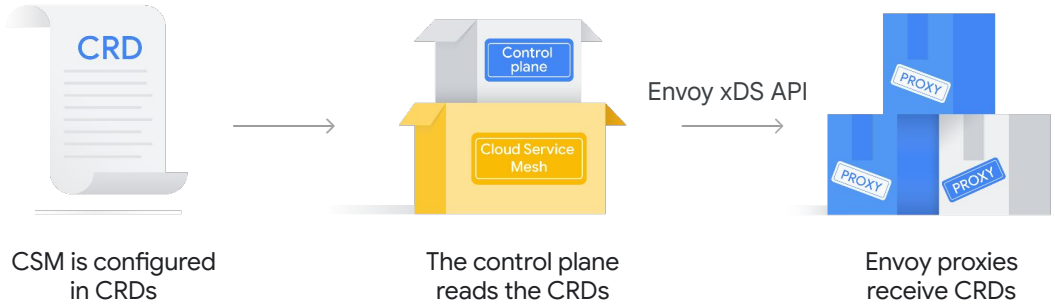
06 Network resilience and testing

Google Cloud

In this section, you learn to configure Cloud Service Mesh with Istio API resources.

Proprietary + Confidential

Cloud Service Mesh CRDs and the Envoy xDS API



Google Cloud

Cloud Service Mesh is configured in YAML files called Kubernetes Custom Resource Definitions (CRDs).

The CSM control plane component reads the CRDs and shares them with the Envoy proxies via the Envoy xDS API.

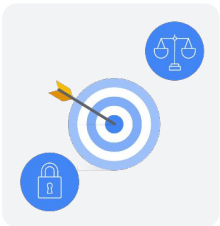
The Envoy xDS API is a set of protocols used by Envoy proxies or proxless gRPC to dynamically fetch configurations from the control plane.

Let's explore the Istio API resources and their CRDs.

Proprietary + Confidential

There are five primary Istio CRDs

VirtualService



DestinationRule

Gateway

ServiceEntry

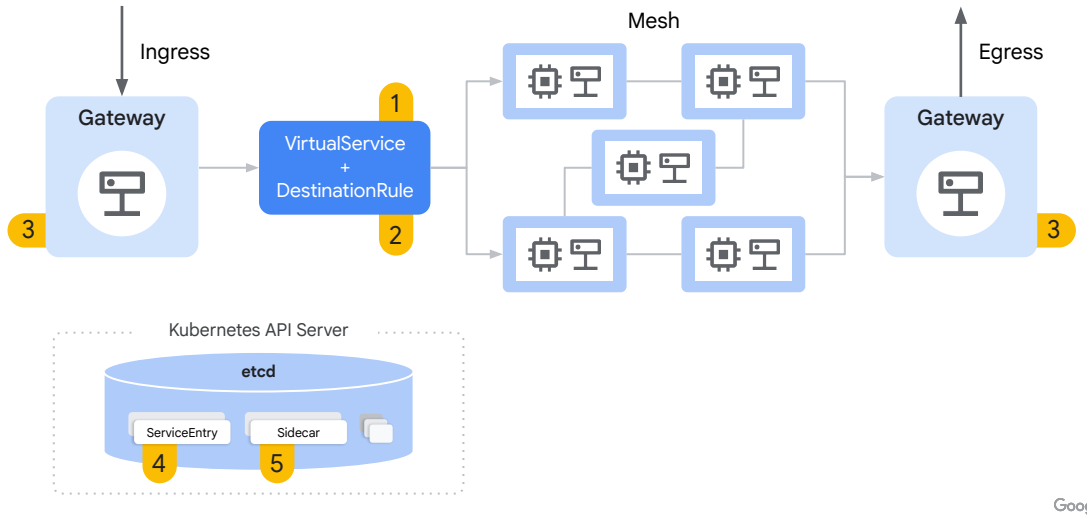
Sidecar

Google Cloud

There are five primary Istio CRDs: VirtualService, DestinationRule, Gateway, ServiceEntry, and Sidecar.

Proprietary + Confidential

Traffic management API resources



Google Cloud

This diagram displays where each of these API resources fits into the traffic management architecture.

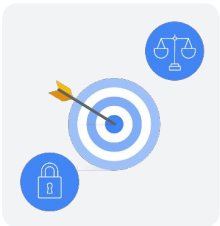
Proprietary + Confidential

VirtualServices and DestinationRules

Where

How

VirtualService



DestinationRule

Gateway

ServiceEntry

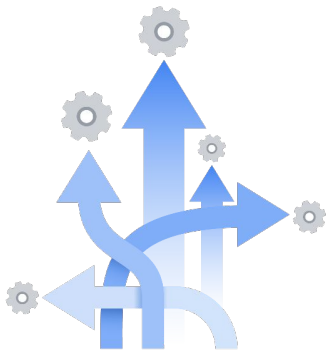
Sidecar

Google Cloud

You can think of VirtualService as the where and DestinationRule as the how.

Proprietary + Confidential

VirtualService



- Defines how requests for a service are routed within an Istio service mesh.
- Can use routing rules to determine the destination of a request.
- Can use rules to route traffic based on elements like request headers, target host name, and URL path.
- Can use target destinations as a service in the mesh registry using proxies, a subset of the service (specific group of pods), or a non-proxied service registered via a ServiceEntry.

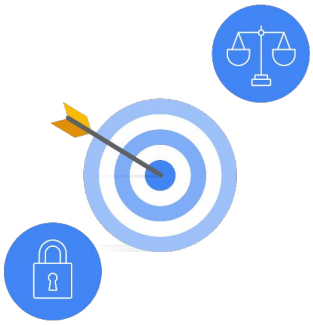
Google Cloud

VirtualService:

- Defines how requests for a service are routed within an Istio service mesh.
- Can use routing rules to determine the destination of a request.
- Can use rules to route traffic based on elements like request headers, target host name, and URL path.
- Can use target destinations as a service in the mesh registry using proxies, a subset of the service (specific group of pods), or a non-proxied service registered via a ServiceEntry.

Proprietary + Confidential

DestinationRule



- Defines how traffic aimed at a particular destination gets handled.
- Can break a single service into multiple subsets (subcollections of pods using labels to select). The VirtualService can then route to the subset.
- Can specify load balancing behavior within the destination.
- Can specify TLS security mode, circuit breaker settings, and other service-level properties.

Google Cloud

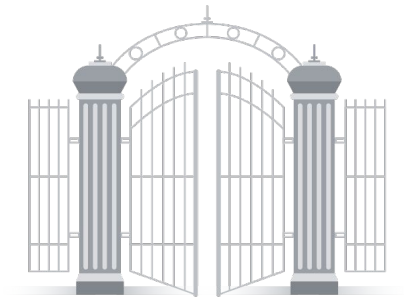
Destination Rule:

- Defines how traffic aimed at a particular destination gets handled.
- Can break a single service into multiple subsets (subcollections of pods using labels to select). The VirtualService can then route to the subset.
- Can specify load balancing behavior within the destination.
- Can specify TLS security mode, circuit breaker settings, and other service-level properties.

A request coming from a client will be routed using both the VirtualService settings (to pick a destination) and DestinationRule settings (to select how to connect to that destination).

Proprietary + Confidential

Gateway



- There are two types of gateways:
  - **Ingress gateways** manage incoming traffic.
  - **Egress gateways** manage outgoing traffic.
- Gateway configuration settings are applied on Envoy proxy pods running on the edge of the mesh.

Google Cloud

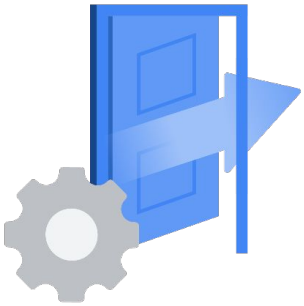
Next, let's define Gateways.

There are two types of Gateways, which are used to manage inbound and outbound traffic for the mesh: ingress gateways for managing incoming traffic, and egress gateways for managing outgoing traffic.

Gateway configuration settings are applied to a deployment on Envoy proxy pods running on the edge of the mesh, not as sidecars to particular workloads.

Proprietary + Confidential

ServiceEntry



- Commonly used to enable requests to services outside of Cloud Service Mesh.
- Useful for accessing external APIs or integrating with legacy services that are not part of the mesh.

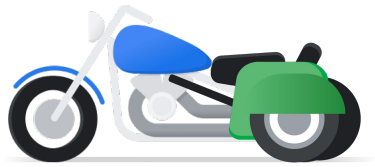
Google Cloud

The next API resource is the ServiceEntry, which is commonly used to enable requests to services outside of an Cloud service mesh.

ServiceEntry is useful for accessing external APIs or integrating with legacy services that are not part of the mesh.

Proprietary + Confidential

Sidecar



- Used to fine-tune Envoy proxy settings.
- They are configured to accept traffic on all ports used by a workload by default.
- They can reach every workload in the mesh.

Google Cloud

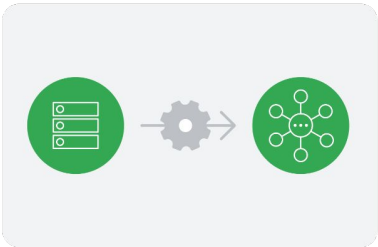
Finally, Sidecars are used to fine-tune Envoy proxy settings.

By default, Sidecar proxies are configured to accept traffic on all ports used by a workload, and can reach every workload in the mesh.

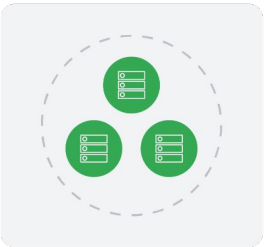
You can use Sidecar configurations to limit protocols, ports, or accessible services in the mesh.

Proprietary + Confidential

WorkloadEntry and WorkloadGroup



WorkloadEntry



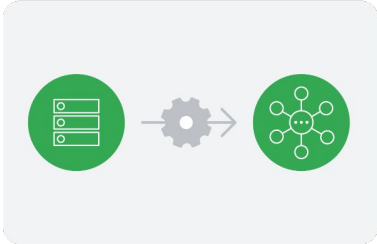
WorkloadGroup

Google Cloud

Two other common API resources are WorkloadEntry and WorkloadGroup.

Proprietary + Confidential

WorkloadEntry



WorkloadEntry

- Used to onboard non-Kubernetes workloads into the mesh.
- A WorkloadEntry uses a ServiceEntry to select workloads and provide the service definition.

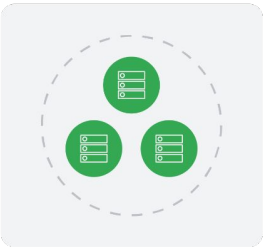
Google Cloud

WorkloadEntry configurations are used to onboard non-Kubernetes workloads into the mesh. Examples of non-Kubernetes workloads include virtual machines and bare metal servers.

A WorkloadEntry uses a ServiceEntry to select workloads and provide the service definition.

Proprietary + Confidential

WorkloadGroup



WorkloadGroup

- A collection of workload instances.
- Designed for non-Kubernetes workloads.
- Replicates the sidecar injection and deployment model used in Kubernetes to bootstrap Istio proxies.

Google Cloud

WorkloadGroup describes a collection of workload instances.

WorkloadGroup is designed for non-Kubernetes workloads, replicating the sidecar injection and deployment model used in Kubernetes to bootstrap Istio proxies.

Proprietary + Confidential

Cloud Service Mesh Routing

01 Configuring Cloud Service Mesh with Istio API resources

02 Configuring VirtualService and DestinationRule

03 Configuring ServiceEntry

04 Configuring a Gateway

05 Configuring a WorkloadEntry and WorkloadGroup

06 Network resilience and testing

Google Cloud

Let’s explore how to configure VirtualService and DestinationRule.

Proprietary + Confidential

Configure traffic behavior with a VirtualService

- Routing rules apply to user-addressable destinations, specified as hosts. These hosts can be:
  - IP addresses.
  - DNS names.
  - Kubernetes Services.
- Use wildcard (*) prefixes to create a single set of routing rules for all matching services.
- Routing rules are evaluated in sequential order from top to bottom.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
    - reviews
  http:
  - match:
    - headers:
        end-user:
          exact: Jason
      route:
      - destination:
          host: reviews
          subset: v2
    - destination:
        host: reviews
        subset: v3
```

Google Cloud

Round-robin is the default load balancing method used by Envoy proxies within a service.

An alternative to the round-robin approach is to use VirtualService to route traffic based on workload characteristics, like different versions for A/B testing or directing traffic for internal users.

This fine-grained control over traffic routing is achieved by specifying hostnames within VirtualService.

VirtualService hostnames can be IP addresses, DNS names, or, if VirtualService is in the same namespace, Kubernetes short names.

Kubernetes short names can result in misconfigurations, so it's recommended that you use a fully qualified host name in production.

Use wildcard (*) prefixes, for example, *.myapp.com, to create a single set of routing rules for all matching services.

VirtualService hosts are virtual destinations and do not have to be part of the Istio service registry. Traffic can be modeled for virtual hosts even if they do not have routable entries inside the mesh.

VirtualService rules and conditions are evaluated in sequential order. In this example,

requests that contain the user Jason are sent to v2, while all traffic that doesn't match this condition is sent to v3.

Proprietary + Confidential

Traffic routing, faults, and mirroring

- Traffic routing options:
  - HTTP headers
  - URIs
  - Ports
  - Source labels
- Supported conditional syntax:
  - Exact
  - Prefix
  - Regex
- Faults test the resilience of your application.
- Traffic mirroring tests shadow deployments and detects network intrusions.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: ratings-route
spec:
  hosts:
    - ratings
  http:
  - match:
    - headers:
        user-agent:
          regex: ^(.*?;)?(iPhone)(;.*)?$
      route:
      - destination:
          host: ratings-iPhone
```

Google Cloud

You can choose from several different VirtualService routing options, including HTTP headers, URIs, ports, and source labels.

In this example, conditional syntax with a regex is used to inspect the user-agent HTTP header. iPhone users are then directed to a specific service.

You can also use VirtualService faults to test the resilience of your application and measure performance in unstable conditions.

Finally, traffic mirroring is a technique used to test shadow deployments and detect network intrusions.

Proprietary + Confidential

Route traffic with a DestinationRule

- DestinationRule defines policies that apply to traffic after it's been routed by the VirtualService.
- Use destination rules to specify named service subsets, such as grouping services by version.
- Subsets are defined based on one or more Kubernetes labels.

```yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: bookinfo-ratings-port
spec:
  host: ratings.prod.svc.cluster.local
  trafficPolicy:
    loadBalancer:
      simple: LEAST_REQUEST
      subsets:
      - name: v2
        labels:
          version: v2
      - name: v3
        labels:
          version: v3
        trafficPolicy:
          loadBalancer:
            simple: ROUND_ROBIN
```

Google Cloud

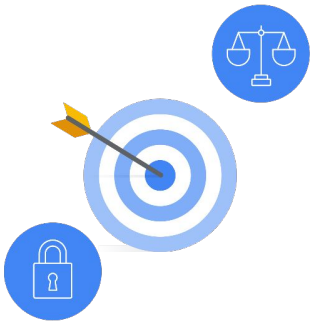
Typically, a VirtualService is coupled with a DestinationRule that determines how traffic is routed. DestinationRules are used to specify named service subsets. For example, you can use them to group a service's instances by version. These subsets can then be used in VirtualService routing rules to direct traffic to specific instances of your services.

Subsets are defined based on one or more labels, which in Kubernetes are key/value pairs attached to objects such as pods.

These labels are defined as metadata in the Deployment of a Kubernetes service.

Proprietary + Confidential

DestinationRules offer different routing strategies



- Load balancing options (round-robin, random, least request, and pass-through).
- Session affinity to consistently route clients to the same service instance.
- Circuit-breaking to limit the impact of failures and latency spikes.

Google Cloud

DestinationRules offer different routing strategies. Strategies include:

- Load balancing options (round-robin, random, least request, and pass-through)
- Configuring session affinity so that clients are consistently routed to the same service instance.
- Configuring circuit-breaking to limit the impact of failures and latency spikes.

Proprietary + Confidential

Traffic splitting

```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: service-b
spec:
  hosts:
    - service-b
  http:
  - route:
    - destination:
        host: service-b
        subset: old-service
      weight: 95
    - destination:
        host: service-b
        subset: new-service
      weight: 5
```

```yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: service-b-destination
spec:
  host: service-b
  trafficPolicy:
    loadBalancer:
      subsets:
      - name: old-service
        labels:
          version: v1
      - name: new-service
        labels:
          version: v2
```

Google Cloud

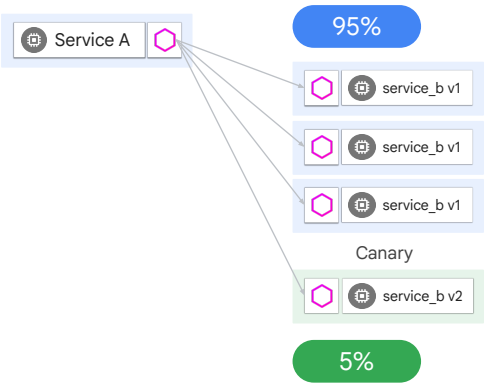
Including multiple destinations in the route attribute lets you split traffic with your VirtualService. These destinations are defined as subsets. For example, a service might have deployment strategy subsets like canary, A/B, or blue-green.

The labels in the DestinationRule must match your pod labels for this configuration to take effect, and the subset field in the VirtualService and DestinationRule must match.

Proprietary + Confidential

Set traffic splitting percentages

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: service-b
spec:
  hosts:
    - service-b
  http:
  - route:
    - destination:
        host: service-b
        subset: old-service
      weight: 95
    - destination:
        host: service-b
        subset: new-service
      weight: 5
```



Google Cloud

Rather than basing traffic splitting on the number of compute instances within the infrastructure, you can leverage a VirtualService's traffic splitting functionality to set specific traffic percentages. This lets you control traffic independently of the number of compute instances in each subset, resulting in traffic management that is decoupled from infrastructure scaling.

Proprietary + Confidential

Cloud Service Mesh Routing

01 Configuring Cloud Service Mesh with Istio API resources

02 Configuring VirtualService and DestinationRule

03 Configuring ServiceEntry

04 Configuring a Gateway

05 Configuring a WorkloadEntry and WorkloadGroup

06 Network resilience and testing

Google Cloud

Next, let’s explore ServiceEntry.

Proprietary + Confidential

Manage traffic for services running outside the mesh

- ServiceEntry adds an entry to the service registry that's maintained by the mesh.
- Envoy proxies can send traffic to the service as if it was a service in your mesh.

```
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: svc-entry
spec:
  hosts:
    - ext-svc.example.com
  ports:
  - number: 443
    name: https
    protocol: HTTPS
  location: MESH_EXTERNAL
  resolution: DNS
```

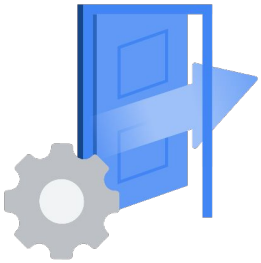
Google Cloud

A ServiceEntry adds an entry to the service registry that's maintained by the mesh.

After the entry has been added, the Envoy proxies can send traffic to the service as if it was a service in your mesh.

Proprietary + Confidential

ServiceEntry



- Redirect and forward traffic for external destinations, like APIs consumed from the web or traffic to services in legacy infrastructure.
- Define retry, timeout, and fault injection policies for external destinations.
- Add VMs to your mesh and run services in them.
- Configure a multi-cluster Istio mesh by logically adding services from different clusters.

Google Cloud

ServiceEntry enables you to manage traffic for services running outside of the mesh. You can:

- Redirect and forward traffic for external destinations, like APIs consumed from the web or traffic to services in legacy infrastructure.
- Define retry, timeout, and fault injection policies for external destinations.
- Add VMs to your mesh and run services in them.
- Configure a multi-cluster Istio mesh by logically adding services from different clusters.

By default, Cloud Service Mesh configures Envoy proxies to pass through requests to unknown services, meaning service mesh features cannot be used to control traffic to these unregistered destinations in the mesh.

Proprietary + Confidential

Cloud Service Mesh Routing

01 Configuring Cloud Service Mesh with Istio API resources

02 Configuring VirtualService and DestinationRule

03 Configuring ServiceEntry

04 Configuring a Gateway

05 Configuring a WorkloadEntry and WorkloadGroup

06 Network resilience and testing

Google Cloud

Along with support for Kubernetes Ingress, Cloud Service Mesh offers Gateways to manage inbound and outbound traffic to the mesh.

Proprietary + Confidential

Istio Gateways are based on the Kubernetes Gateway API

- Use Istio Gateways for more granular control over load balancing, TLS settings, and routing rules.
- Gateway configurations applied to standalone Envoy proxies at the edge of the mesh to configure L4-L7 properties such as ports and TLS settings.

```yaml
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: bookinfo-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    hosts:
    - "*"
    tls:
      mode: SIMPLE
      credeentialName: ext-cert
```

Google Cloud

Gateways in Istio are based on the Kubernetes Gateway API, which standardizes ingress traffic management.

Istio Gateways provide more extensive customization and flexibility than Kubernetes Ingress, allowing greater granularity when configuring load balancing properties, TLS settings, and routing rules.

Gateways are applied to standalone Envoy proxies at the mesh edge, not to sidecar proxies, and they configure Layer 4-6 load balancing (ports, TLS).

VirtualServices handle Layer 7 (application-level) routing bound to the Gateway.

Proprietary + Confidential

There are three main types of Gateways

Ingress gateway

A load balancer at the edge of the mesh that receives HTTP/TCP connections.

Egress gateway

Represents a dedicated exit node for the traffic leaving the mesh.

East-west gateway

Proxies for service workloads to communicate across cluster.

Google Cloud

There are three main types of Gateways:

Ingress gateways are load balancers located at the edge of the mesh. They receive HTTP/TCP connections.

Egress gateways represent dedicated exit nodes for traffic leaving the mesh, controlling external network access. This enables secure control of egress traffic to enhance the security of your mesh.

East-west gateways are proxies for cross-cluster traffic in multi-primary meshes. East-west gateways are essential for managing inter-cluster communication and applying policies to traffic flowing between clusters.

Proprietary + Confidential

Bind your VirtualService to a Gateway

```yaml
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: bookinfo-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    hosts:
    - "*"
    tls:
      mode: SIMPLE
      credeentialName: ext-cert
```

```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: bookinfo
spec:
  hosts:
    - "*"
  gateways:
  - bookinfo-gateway
  http:
    - match:
        uri:
          exact: /productpage
      route:
      - destination:
          host: productpage
          port:
            number: 9080
```

Google Cloud

To route inbound traffic, bind a VirtualService to a Gateway.

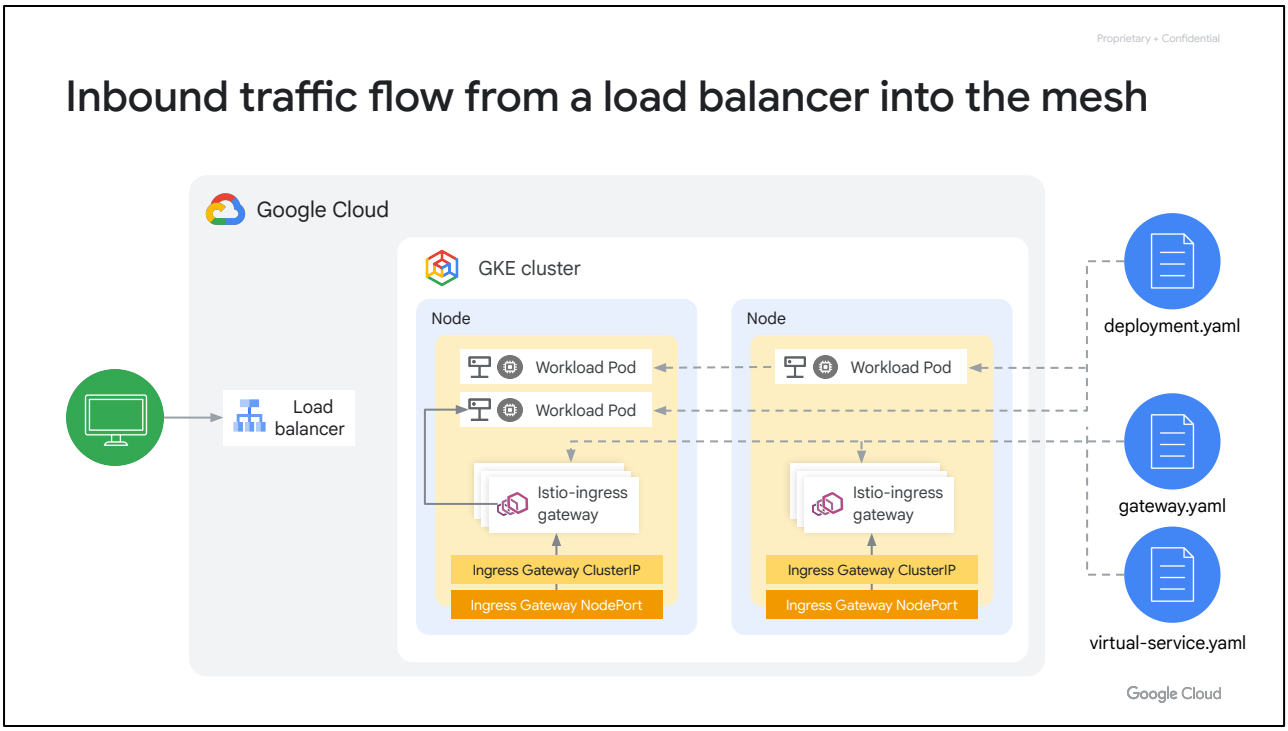
Ingress gateways can enable HTTPS connections and link to external certificates via a VirtualService.

The gateways field in a VirtualService links to the specific gateway, in this example “bookinfo-gateway”.

The host field is defined by an asterisk, meaning the VirtualService accepts traffic from any destination.

After it’s evaluated, the VirtualService routes traffic based on the route attribute.

The host attribute on spec.http.match.route.destination.host refers to the Kubernetes service.



The istio-ingressgateway is a Kubernetes LoadBalancer.

A Google Cloud load balancer is created along with an ingress gateway.

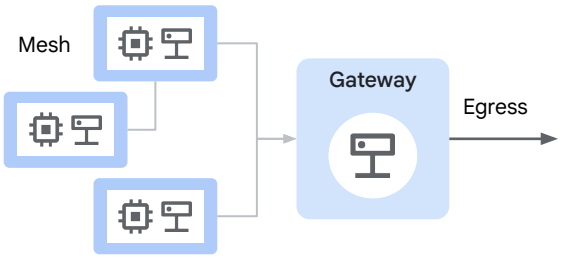
Inbound traffic flows from a load balancer into the mesh. The load balancer forwards traffic to a NodePort and then to the ingress gateway pod.

The ingress gateway deployment has a single Envoy container configured with VirtualServices and DestinationRules.

The ingress gateway's Envoy processes requests and forwards them to the appropriate destination.

Proprietary + Confidential

Egress Gateways control traffic exiting the mesh



- Egress gateways provide control over traffic exiting the service mesh.
- They can be used to enforce security policies for outbound traffic.
- Egress Gateways are useful for notes that lack public IPs and are unable to access the internet.

Google Cloud

So we explored ingress gateways, but what are egress gateways?

Egress gateways provide control over traffic exiting the service mesh. They can be used to enforce security policies for outbound traffic. For example, traffic may need to pass through monitored notes, potentially on separate machines.

Egress Gateways are also used in scenarios where the nodes of an application lack public IPs and are unable to access the internet.

Egress traffic can be routed through the egress gateway and assigned public IPs, enabling applications to access external services securely.

Proprietary + Confidential

Configure an egress gateway

01 Add a ServiceEntry for the external location.

02 Create a Gateway resource using the same host as the ServiceEntry.

03 Configure the Gateway to allow outbound traffic.

Optional Configure routing or load balancing.

Google Cloud

To configure an egress gateway, you need to perform the following steps:

First, add a ServiceEntry for the external location (for example, `sheets.google.com`).

Then create a Gateway resource using the same host as the ServiceEntry.

Configure the Gateway to allow outbound traffic.

Optionally, configure routing or load balancing using VirtualServices and DestinationRules.

Proprietary + Confidential

Cloud Service Mesh Routing

01 Configuring Cloud Service Mesh with Istio API resources

02 Configuring VirtualService and DestinationRule

03 Configuring ServiceEntry

04 Configuring a Gateway

05 Configuring a WorkloadEntry and WorkloadGroup

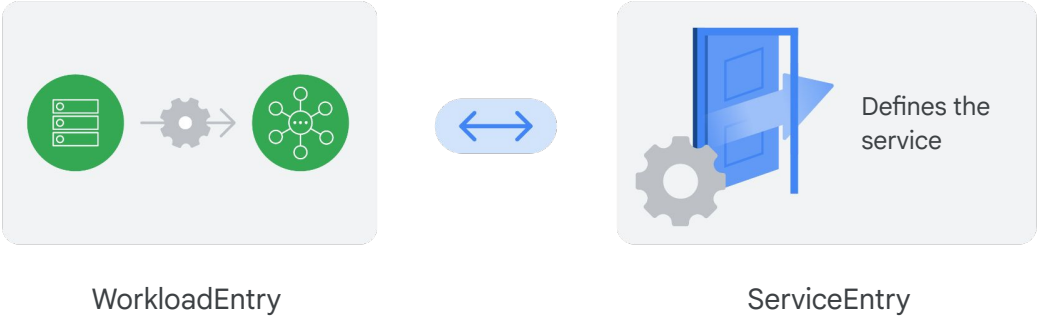
06 Network resilience and testing

Google Cloud

In this section, you explore WorkloadEntry and WorkloadGroup. WorkloadEntry enables the integration of non-Kubernetes workloads by defining the workload properties, such as its address, service account, and labels. In Istio, WorkloadGroup provides a mechanism to manage collections of non-Kubernetes workloads by defining their bootstrap properties.

Proprietary + Confidential

WorkloadEntry must have a corresponding ServiceEntry



Google Cloud

Let's begin by exploring how to configure WorkloadEntry.

A WorkloadEntry must have a corresponding ServiceEntry, which defines the service itself (hostnames, ports). ServiceEntry uses label selectors to associate the service with the WorkloadEntry.

Proprietary + Confidential

Centralized workload monitoring and management

Google Cloud

When a workload connects to Istio, its status is updated, containing its health and other details, enabling centralized monitoring and management of all workloads in the mesh.

Proprietary + Confidential

WorkloadEntry and ServiceEntry for a VM with a static IP address

```yaml
apiVersion: networking.istio.io/v1
kind: WorkloadEntry
metadata:
  name: details-svc
spec:
  serviceAccount: details-legacy
  address: 2.2.2.2
  labels:
    app: details-legacy
    instance-id: vm1
```

```yaml
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: details-svc
spec:
  hosts:
  - details.bookinfo.com
  location: MESH_INTERNAL
  ports:
  - number: 80
    name: http
    protocol: HTTP
    targetPort: 8080
  resolution: STATIC
  workloadSelector:
    labels:
      app: details-legacy
```

Google Cloud

This is an example of a WorkloadEntry for a VM with a static IP address. The VM uses a `MESH_INTERNAL` ServiceEntry with the resolution defined as `STATIC`.

Traffic is routed through sidecars, with Istio mutual TLS securing communication.

Proprietary + Confidential

WorkloadEntry and ServiceEntry for a VM with a qualified DNS name

```yaml
apiVersion: networking.istio.io/v1
kind: WorkloadEntry
metadata:
  name: details-svc
spec:
 serviceAccount: details-legacy
  address: vm1.vpc01.corp.net
  labels:
    app: details-legacy
    instance-id: vm1
```

```yaml
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: details-svc
spec:
  hosts:
  - details.bookinfo.com
  location: MESH_INTERNAL
  ports:
  - number: 80
    name: http
    protocol: HTTP
    targetPort: 8080
  resolution: DNS
  workloadSelector:
    labels:
      app: details-legacy
```

Google Cloud

In this example, we are using a fully qualified DNS name for the VM.

In this case, the ServiceEntry's resolution mode is set to DNS, enabling client-side sidecars to dynamically resolve the DNS name.

Proprietary + Confidential

Use WorkloadGroup to manage non-Kubernetes workloads

- Manage collections of non-Kubernetes workloads by defining their bootstrap properties.
- They mimic Kubernetes' sidecar injection and deployment models.
- WorkloadGroup is essentially a template for WorkloadEntry resources.

Google Cloud

Next, let's transition to WorkloadGroups. You can use **WorkloadGroup** to manage collections of non-Kubernetes workloads by defining their bootstrap properties. They mimic Kubernetes' sidecar injection and deployment models.

WorkloadGroup is essentially a template for WorkloadEntry resources.

Proprietary + Confidential

WorkloadGroup example

```yaml
apiVersion: networking.istio.io/v1
kind: WorkloadGroup
metadata:
  name: reviews
  namespace: bookinfo
spec:
  metadata:
    labels:
      app.kubernetes.io/name: reviews
      app.kubernetes.io/version: "1.3.4"
  template:
    ports:
      grpc: 3550
      http: 8080
serviceAccount: default
...
```

```yaml
...

probe:
    initialDelaySeconds: 5
    timeoutSeconds: 3
    periodSeconds: 4
    successThreshold: 3
    failureThreshold: 3
    httpGet:
     path: /foo/bar
     host: 127.0.0.1
     port: 3100
     scheme: HTTPS
     httpHeaders:
     - name: Lit-Header
       value: Im-The-Best
```

Google Cloud

A WorkloadGroup defines metadata, like labels, and a template specifying ports and service accounts for associated workloads.

This ensures consistent configuration across multiple instances. Health probes can also be defined, allowing Istio to monitor workload health.

This example defines a WorkloadGroup named “reviews” in the "bookinfo" namespace. The WorkloadGroup has specific labels, ports (gRPC and HTTP), and a service account. It also includes a health probe configuration.

The WorkloadGroup focuses on workload configuration and bootstrapping, not service registry. Hostname configuration is managed by ServiceEntry.

WorkloadGroup uses common properties across WorkloadEntry resources to simplify management and ensure consistency.

Proprietary + Confidential

Cloud Service Mesh Routing

01 Configuring Cloud Service Mesh with Istio API resources

02 Configuring VirtualService and DestinationRule

03 Configuring ServiceEntry

04 Configuring a Gateway

05 Configuring a WorkloadEntry and WorkloadGroup

06 Network resilience and testing

Google Cloud

In this section, you explore WorkloadEntry and WorkloadGroup. WorkloadEntry enables the integration of non-Kubernetes workloads by defining the workload properties, such as its address, service account, and labels. In Istio, WorkloadGroup provides a mechanism to manage collections of non-Kubernetes workloads by defining their bootstrap properties.

Proprietary + Confidential

Use request timeouts to avoid requests to hang infinitely

- Limit the time that an Envoy proxy should wait for replies from a given service.
- Ensure that calls succeed or fail within a predictable timeframe.
- Set up a timeout in the VirtualService because Envoy timeouts for HTTP requests are disabled by default.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: service-b
spec:
  hosts:
  - service-b.default.svc.cluster.local
  http:
  - route:
    - destination:
        host:
service-b.default.svc.cluster.local

        subset: v1
    timeout: 10s
```

Google Cloud

A **timeout** is used to set the amount of time that an Envoy proxy should wait for replies from a given service.

This ensures that services don't wait for replies indefinitely and that calls succeed or fail within a predictable timeframe.

If set too long, timeouts cause excessive latency; if set too short, calls may fail unnecessarily.

This example specifies a 10-second timeout for calls to the service_b service.

VirtualService let you easily adjust timeouts dynamically on a per-service basis without having to edit your service code.

Envoy timeouts for HTTP requests are disabled by default.

Proprietary + Confidential

Use retries to set the maximum attempts to call a service

- Retries can improve service availability and application performance.
- They can solve temporary issues like temporarily overloading a service or network.
- Interval between retries (25ms+) is variable and determined by Cloud Service Mesh.
- By default, HTTP requests retry twice before returning an error.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews.default.svc.cluster.local

  http:
  - route:
    - destination:
        host:
reviews.default.svc.cluster.local
        subset:
 v1
    retries:
      attempts: 3
      perTryTimeout: 2s
```

Google Cloud

You can specify the maximum number of times an Envoy proxy can attempt to connect to a service with the retry setting.

Retries can improve service availability and application performance by ensuring that temporary errors don't cause permanent call failures.

The interval between retries (25 ms+) is variable and determined automatically by Cloud Service Mesh to avoid overwhelming the called services with requests.

By default, HTTP requests retry twice before returning an error.

This example configures a maximum of three retries to connect to this service subset after an initial call failure, each with a two-second timeout.

Proprietary + Confidential

Improve resiliency with circuit breakers

- Prevent connections to an overloaded or failing host.
- Limit the effect of outliers or faulty clients on services.
- Circuit breakers provide two main mechanisms:
  - Connection pool size
  - Outlier detection

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
spec:
  hosts: reviews
  subsets:
  - name: v1
  - labels:
      version: v1
    trafficPolicy:
      connectionPool:
        tcp:
          maxConnections: 100
      outlierDetection:
        consecutive5xxErrors: 3
        interval: 1s
        baseEjectionTime: 2m
        maxEjectionPercent: 50
```

Google Cloud

Circuit breakers increase the resilience of microservice-based applications.

In a circuit breaker, you set call limits to individual hosts within a service, like the amount of concurrent connections or failed calls to the host.

Once the limit has been reached, the circuit breaker "trips" and stops further connections to that host.

Circuit breakers fail quickly and keep clients from trying to connect to an overloaded or failing host.

Proprietary + Confidential

Circuit breaker thresholds are defined in DestinationRules

```yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews.default.svc.cluster.local
  subsets:
  - name: v1
    labels:
      version: v1

    trafficPolicy:
      connectionPool:
        tcp:
          maxConnections: 100
```

Google Cloud

Circuit breaker thresholds are defined in DestinationRules, and settings are applied to each individual host in the service.

In this example, the amount of concurrent connections for the reviews service workloads of the v1 subset is limited to 100.

Proprietary + Confidential

Use outlier detection to detect and evict unhealthy hosts

```yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews.default.svc.cluster.local
  subsets:
  - name: v1
    labels:
      version: v1

    trafficPolicy:
      outlierDetection:
        consecutiveErrors: 3
        interval: 1s
        baseEjectionTime: 2m
        maxEjectionPercent: 50
```

Google Cloud

You can use **outlier detection** settings to detect and evict unhealthy hosts from the load balancing pool.

In this example, if the reviews service returns three consecutive 500 errors in a 1-second interval, the service will be ejected for 2 minutes.

Up to 50% of all services can be evicted in this example.

Proprietary + Confidential

Fault injections

Use fault injections like delay and abort to test how an application responds to failure.

```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - route:
    - destination:
        host: ratings
        subset: v1
    fault:
      delay:
        percentage:
          value: 0.1
        fixedDelay: 5s
      abort:
        percentage:
          value: 0.1
        httpStatus: 400
```

Google Cloud

Fault injections like delay and abort simulate scenarios like network delays, service overloads, and HTTP errors to test how an application responds to failure.

Proprietary + Confidential

Delays

Delay requests before forwarding or simulating failures like network issues or overloaded upstream service.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews.default.svc.cluster.local
  http:

  - fault:
      delay:
        fixedDelay: 5s
        percentage:
          value: 0.1
    route:
    - destination:
        host:
reviews.default.svc.cluster.local
        subset: v1
```

Google Cloud

Delay requests are applied before forwarding or simulating failures like network issues or overloaded upstream service.

This example introduces a 5-second delay in one out of every 1000 requests.

The fixedDelay field indicates the amount of delay in seconds. The optional percentage field can be used to only delay a certain percentage of requests.

If left unspecified, all requests will be delayed.

Proprietary + Confidential

Delays

Abort HTTP request attempts and returns error codes to the downstream service, simulating an upstream service failure.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews.default.svc.cluster.local
  http:

  - fault:
      abort:
        httpStatus: 400
        percentage:
          value: 100
    route:
    - destination:
        host:
reviews.default.svc.cluster.local
        subset: v1
```

Google Cloud

Abort HTTP request attempts and returns error codes to the downstream service, simulating an upstream service failure.

This example introduces a 400 HTTP status code to return to the caller.

The optional percentage field can be used to only abort a certain percentage of requests. If not specified, all requests are aborted.

Delay and abort faults are independent of one another, even if both are specified simultaneously.

Proprietary + Confidential

Traffic mirroring

- Traffic mirroring (shadowing) minimizes risk when pushing changes to production.
- Sends a copy of live traffic to a mirrored service.
- Detect network intrusions, discover unauthorized traffic paths, or identify unsecure plain text traffic.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - route:
    - destination:
        host: ratings
        subset: v1
    mirror:
      host: ratings
      subset: v2
    mirrorPercent: 100
```

Google Cloud

Traffic mirroring, also known as **shadowing**, minimizes risk when pushing changes to production.

Mirroring sends a copy of live traffic to a mirrored service. The mirrored traffic happens out of band of the critical request path for the primary service.

Traffic analysis is another interesting use case.

Send a copy of your live traffic to an analytics service to detect network intrusions, discover unauthorized traffic paths, or identify unsecure plain text traffic.

In this example, 100% of the traffic going to v1 to v2 is mirrored.